



Temporal Databases: Concepts and Applications



Managing Data That Changes Over Time

Course: Database Management Systems (BTS30102)

Presented by: Sayantan Bharati | BWU/BTS/25/503 | Section B

Programme: B.Tech (CSE), Batch 2025, AY 2026-27 (Odd Semester 3)

Faculty: Ataul Hoque Mandal

Department of Computer Science & Engineering, Brainware University

Date: 13-10-2026

Introduction : Why Do We Need Time in Databases?

- A conventional DBMS keeps only the **current state** : the old value is overwritten.
- But many questions are about the **past**: "What was the salary in 2023?"
- Overwriting data destroys the **audit trail** and the ability to reproduce past decisions.
- **Temporal databases** store time as a first-class part of every fact.
- They answer *when* a fact was true and *when* the database believed it was true.
- Built on standard DBMS ideas : schema, keys, transactions, indexing : extended with time.

Conventional

Emp101 | Salary = 70000

old value **GONE**

Temporal

Emp101 | 50000 | 2020-2023

Emp101 | 70000 | 2023-now

history kept



Presentation Outline

- 1 The Need for Time in Data
- 2 Core Concepts & Time Dimensions
- 3 Temporal Data Models
- 4 Real-World Applications
- 5 SQL:2011 Temporal Features
- 6 Temporal Querying & Interval Algebra
- 7 Implementation & Indexing
- 8 Worked Example : Salary History
- 9 Temporal Databases & Transactions
- 10 Challenges, Best Practices & Future

Why Conventional Databases Fall Short

- Conventional DBMS = **snapshot database**: only the current moment is stored.
- An **UPDATE** or **DELETE** physically destroys the previous value.
- Audit, legal and analytical questions then become impossible or costly.
- Common workarounds are weak: history tables, shadow copies, log mining.
- History tables duplicate schema, drift over time and are hard to query uniformly.
- A temporal DBMS makes history a **built-in, queryable dimension**.

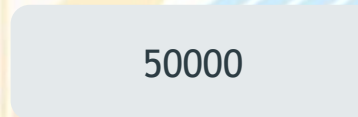
Snapshot DBMS



old value overwritten

70000

Temporal DBMS



70000

2020

2023

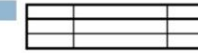
now

The Two Kinds of Time

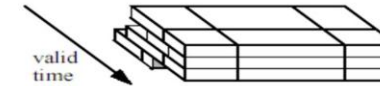
- **Valid time** (application time): when the fact is true in the real world.
- **Transaction time** (system time): when the fact was stored/believed by the DBMS.
- **Bitemporal** databases track *both* at once.
- Examples: "the price was valid in March" vs "the price was recorded on 2 April".
- **Timestamp** = a single instant; **interval/period** = a start and end.
- These dimensions give rise to the **bitemporal cube** of possible questions.

Time Dimension: Temporal Relations

□ Snapshot relation



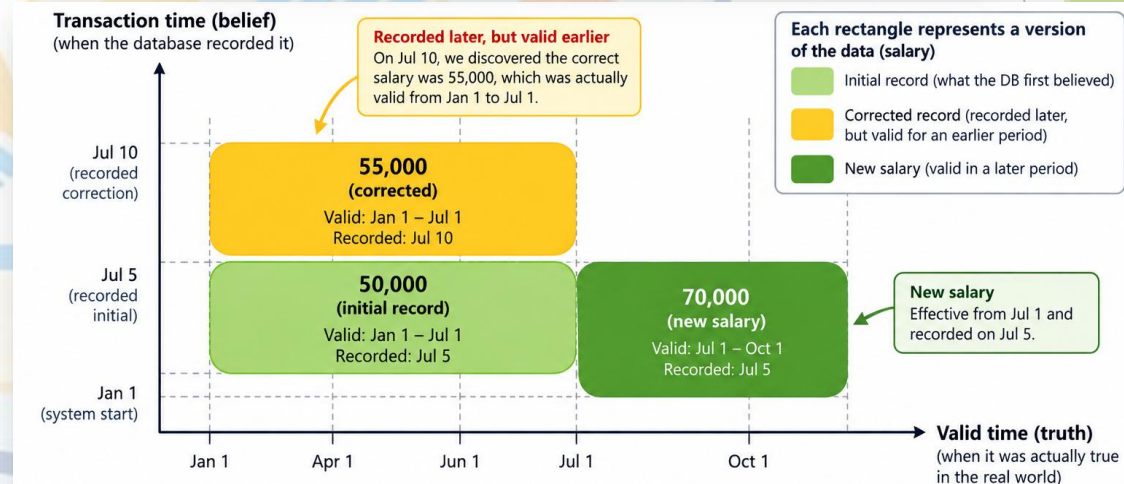
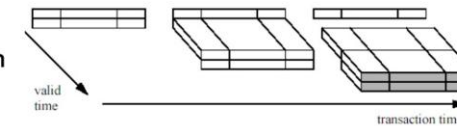
□ Valid-time relation



□ Transaction-time relation



□ Bitemporal relation



Temporal Data Models

- **Snapshot model** : no time; classic DBMS.
- **Rollback / transaction-time model** : versions by when they were stored.
- **Valid-time (historical) model** : versions by real-world validity.
- **Bitemporal model** : both valid and transaction time.
- **SQL:2011** standardized these as **system-versioned** and **application-time period** tables.
- Each model trades storage and complexity for richer history.

Model	Valid time	Trans. time	Typical use
Snapshot	-	-	Current-only apps
Rollback	-	Yes	Auditing, recovery
Valid-time	Yes	-	Contract / price history
Bitemporal	Yes	Yes	Regulatory, finance

Where Temporal Databases Are Used

BF

Banking & finance
balances, interest rates and
transaction histories.

I

Insurance
policies valid over ranges; claims
recorded later.

H

Healthcare
patient conditions valid over time
vs when entered.

AC

Audit & compliance
immutable, queryable audit trails.

SC

Supply chain & telecom
pricing, inventory and billing
history.

DW

Data warehousing
type-2 slowly changing dimensions
are temporal by nature.

Querying Time with SQL:2011

- **System-versioned tables** start with:
`WITH SYSTEM VERSIONING.`
- **Application-time period tables**
declare: `PERIOD FOR valid_time`
`(start_date, end_date).`
- **Query a past state:** `SELECT ... FOR`
`SYSTEM_TIME AS OF TIMESTAMP`
`'2023-06-01'.`
- **Update only a portion of history:**
`UPDATE ... FOR PORTION OF`
`valid_time FROM ... TO ....`
- Temporal primary keys prevent overlapping valid periods.
- This turns history queries into normal SQL instead of hand-written joins.

SQL

```
CREATE TABLE salary (  
  emp_id      INT,  
  amount      DECIMAL(10,2),  
  valid_time  PERIOD FOR valid_time(from_date, to_date),  
  PRIMARY KEY (emp_id,valid_time WITHOUT OVERLAPS)  
) WITH SYSTEM VERSIONING;  
  
-- What did the system store on 1 June 2023?  
SELECT * FROM salary  
  FOR SYSTEM_TIME AS OF TIMESTAMP '2023-06-01';
```

AS OF = the database exactly as it looked on any past date

Asking Questions About Intervals

- Time can be an **instant** (timestamp) or an **interval/period** (start, end).
- **Allen's interval algebra** defines 13 possible relations between two intervals.
- Examples: *before*, *after*, *meets*, *overlaps*, *contains*, *during*, *equals*.
- Temporal SQL extends joins and predicates with these relations.
- Finding overlaps is the most common operation (e.g., "who held the role then?").
- Efficient interval logic is what separates temporal query languages from plain SQL.

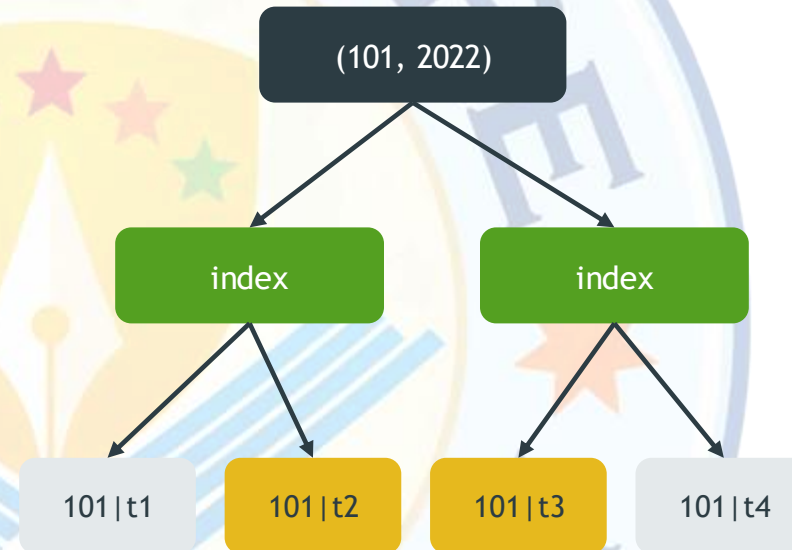
Allen's 13 relations: A (green) vs B (dark)



How Temporal Databases Are Built

- **Tuple versioning:** store one row per version, each with start/end timestamps.
- **Attribute-level timestamping:** timestamp individual changed columns (saves space).
- **Transaction-time** gives an append-only, immutable store : high write throughput.
- Indexing must handle **ranges**, not just points : e.g., interval / temporal B+ trees.
- Common structures: R-trees for intervals, temporal B+ trees, append-only LSM stores.
- MVCC and timestamping (from the concurrency unit) are close cousins of temporal storage.

B+ tree keyed on (id, time)

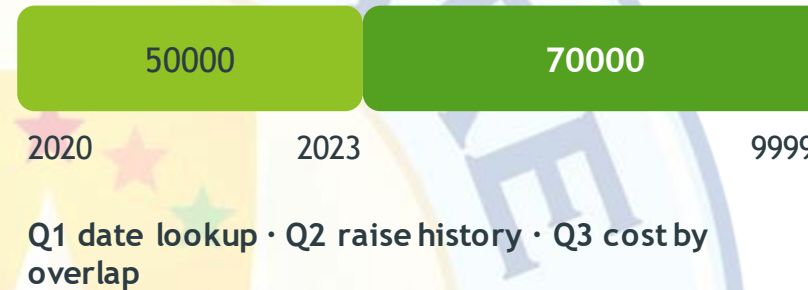


interval scan over highlighted leaves

One row per version: id | value | start | end

Worked Example : Tracking Salaries

- Scenario: an employee's salary changes over time; we must answer past questions.
- Store each version with `from_date`, `to_date` (valid time).
- Query 1: salary on a given date → select the row whose period contains it.
- Query 2: full raise history → order rows by `from_date`.
- Query 3: total cost over a period → join with interval overlap.
- With system versioning, also ask "what did the DB believe on date X?".



```

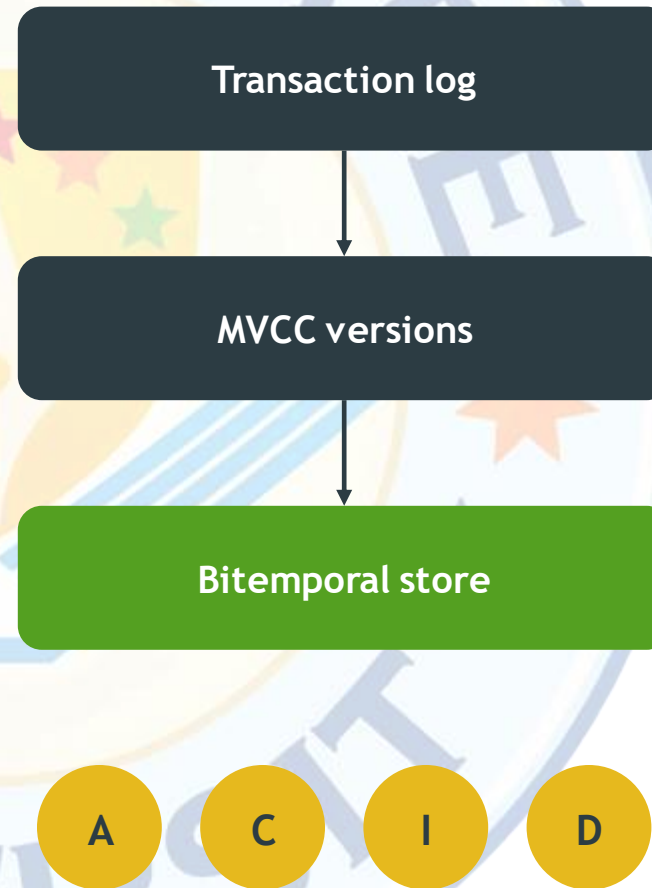
SQL

emp_id  amount  from_date  to_date
101     50000   2020-01-01 2023-01-01
101     70000   2023-01-01 9999-12-31

-- salary on 2021-05-10
SELECT amount FROM salary
WHERE emp_id = 101 AND
      DATE '2021-05-10'
      BETWEEN from_date AND to_date;
  
```

Ties to Transactions & Concurrency

- Transaction time is exactly what a **transaction log / recovery** already records.
- **MVCC** keeps multiple versions of a row : the same versioning idea.
- Timestamp-based schedulers already order operations by time.
- Temporal storage is naturally **append-only**, so it is rollback-friendly.
- Serializability still applies: history must reflect a consistent order of transactions.
- This connects directly to the course's transaction, ACID and recovery modules.



Challenges, Best Practices & Future

- **Storage growth** : every change becomes a permanent version.
- **Performance** : range/interval indexes are costlier than point indexes.
- **Complexity** : developers must reason about two timelines.
- **Privacy** : "right to be forgotten" conflicts with immutable history; needs crypto-shredding.
- **Best practices**: define retention policy, choose the right model, index intervals, separate hot/cold data.
- **Future**: cloud temporal services, ML on history, blockchain-anchored audit trails.

Benefits

- Full audit trail
- Time-travel queries
- Regulatory ready

Challenges

- Storage growth
- Costlier indexes
- Two timelines
- Privacy

Future → cloud temporal services, ML on history, blockchain-anchored audit



References & Thank You

- A. Silberschatz, H. Korth, S. Sudarshan : *Database System Concepts* (7th ed.), McGraw Hill.
- R. Elmasri & S. Navathe : *Fundamentals of Database Systems*, Addison-Wesley.
- ISO/IEC 9075 : **SQL:2011** temporal features (system-versioned & application-time tables).
- R. T. Snodgrass : *Developing Time-Oriented Database Applications in SQL*.
- Course lecture notes : DBMS Modules 1-3 (data models, transactions, storage).

Thank you.

Sayantan Bharati | BWU/BTS/25/503 | Section B